# CSE 309: Theory of Automata
# Computational Complexity

**Shahid Hussain**

shahidhussain@iba.edu.pk

April/May 2024

School of Mathemtics and Computer Science
Institue of Business Administration

# Measuring Complexity

# Measuring Complexity

- Consider $A = \{0^k 1^k : k \geq 0\}$
- $A$ is a decidable language
- The question is "*how much time does a single-tape Turing machine need to decide $A$?*"
- Consider the TM $M_1$ for $A$:

# Measuring Complexity

- Consider $A = \{0^k 1^k : k \geq 0\}$
- $A$ is a decidable language
- The question is "*how much time does a single-tape Turing machine need to decide $A$?*"
- Consider the TM $M_1$ for $A$:

$M_1 =$ "On input string $w$:

1. Scan across the tape and *reject* if a $0$ is found to the right of a $1$
2. Repeat if both $0$'s and $1$'s remain on the tape:
3. Scan across the tape, crossing off a single $0$ and a single $1$
4. If $0$'s still remain after all $1$'s been crossed off, or vice verta, *reject*. Otherwise, *accept*

# Time Complexity

**Time Complexity**

Let $M$ be a deterministic Turing machine that halts on all inputs. The **running time** or **time complexity** of $M$ is the function $f : \mathbb{N} \to \mathbb{N}$ where $f(n)$ is the maximum number of steps that $M$ uses on any input of length $n$. If $f(n)$ is the running time of $M$, we say that $M$ runs in time $f(n)$ and that $M$ is an $f(n)$ time Turing machine.

# Asymptotic Notations

# Bounds

## Asymptotic Upper Bound

Let $f$ and $g$ be functions $f, g : \mathbb{N} \to \mathbb{R}^+$. We say that $f(n) = O(g(n))$ if positive integers $c$ and $n_0$ exists such that for every integer $n \geq n_0$

$$f(n) \leq c \cdot g(n).$$

When $f(n) = O(g(n))$ we say that $g(n)$ is an **upper bound** for $f(n)$ or more precisely, that $g(n)$ is an **asymptotic upper bound** for $f(n)$.

# Bounds

## Small-oh

Let $f$ and $g$ be functions $f, g : \mathbb{N} \to \mathbb{R}^+$. We say that $f(n) = o(g(n))$ if

$$\lim_{n \to \infty} \frac{f(n)}{g(n)} = 0.$$

In other words, $f(n) = o(g(n))$ means that for any real number $c > 0$, a number $n_0$ exists, where $f(n) < c \cdot g(n)$ for all $n \geq n_0$.

# Time Complexity Class

**Time Complexity Class**

Let $t : \mathbb{N} \to \mathbb{R}^+$ be a function. Define the **time complexity class**, $\mathrm{TIME}(t(n))$, to be the collection of all languages that are decidable by an $O(t(n))$ time Turing machine.

# Time Complexity Class

## Time Complexity Class

Let $t : \mathbb{N} \to \mathbb{R}^+$ be a function. Define the **time complexity class**, $\mathrm{TIME}(t(n))$, to be the collection of all languages that are decidable by an $O(t(n))$ time Turing machine.

## Theorem 1

*Let $t(n)$ be a function, where $t(n) \geq n$. Then every $t(n)$ time multitape Turing machine has an equivalent $O(t^2(n))$ time single-tape Turing machine.*

# Time Complexity Class

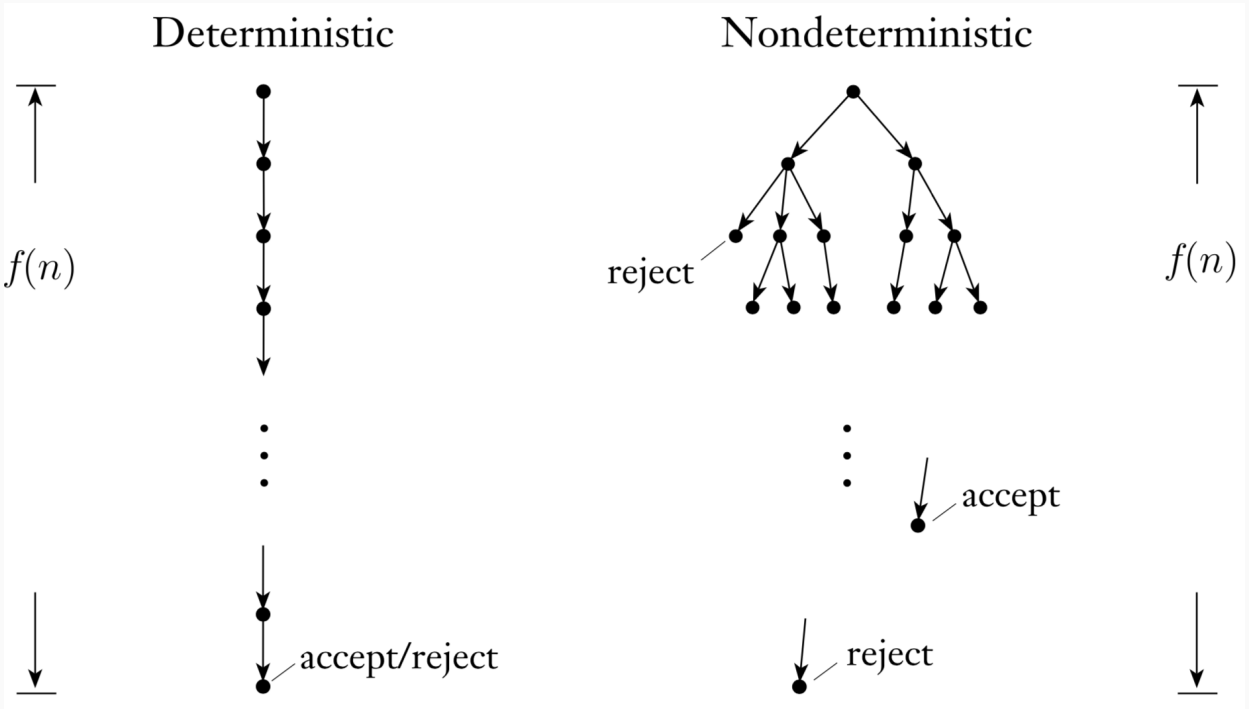
**Running Time for NTM**

Let $N$ be a nondeterministic Turing machine that is a decider. The **running time** of $N$ is the function $f : \mathbb{N} \to \mathbb{N}$, where $f(n)$ is the maximum number of steps that $N$ uses on any branch of its computation on any input of length $n$.

# Time Complexity Class

## Running Time for NTM

Let $N$ be a nondeterministic Turing machine that is a decider. The **running time** of $N$ is the function $f : \mathbb{N} \to \mathbb{N}$, where $f(n)$ is the maximum number of steps that $N$ uses on any branch of its computation on any input of length $n$.



# Complexity Relationships

**Theorem 2**

*Let $t(n)$ be a function, where $t(n) \geq n$. Then every $t(n)$ time nondeterministic single-tape Turing machine has an equivalent $2^{O(t(n))}$ time deterministic single-tape Turing machine.*

# The Class P

# Polynomial Time

- We will consider polynomial differences in running time as small
- For example $n^3$ and $2^n$ are drastically different growth rates
- Exponential algorithms arise when we try to exhaustively search through a space of solutions
- All reasonable computational models are **polynomially equivalent** i.e., any model can simulate other

# The Class P

## The Class P

**P** is the class of languages that are decidable in polynomial time on a deterministic single-tape Turing machine. In other words,

$$\mathbf{P} = \bigcup_k \mathrm{TIME}(n^k).$$

# The Class P

## The Class P

**P** is the class of languages that are decidable in polynomial time on a deterministic single-tape Turing machine. In other words,

$$\mathbf{P} = \bigcup_k \mathrm{TIME}(n^k).$$

1. **P** is invariant for all models of computation that are polynomially equivalent to the deterministic single-tape Turing machine, and
2. **P** roughly corresponds to the class of problems that are realistically solvable on a computer

# Examples of Problems in P

**Theorem 3**

$\text{PATH} \in \boldsymbol{P}$, *where* $G$ *is a directed path*

$$\text{PATH} = \{\langle G, s, t\rangle : G \text{ has a directed path from } s \text{ to } t\}.$$

# Examples of Problems in P

- We say that two numbers $a$ and $b$ are relatively prime if $1$ is the largest integer that evenly divides them both
- For example, $10$ and $21$ are relatively prime, even though neither of them is a prime number by itself, whereas $10$ and $22$ are not relatively prime because both are divisible by $2$
- Let RELPRIME be the problem of testing whether two numbers are relatively prime

**Theorem 4**

RELPRIME $\in$ ***P***.

# The Class NP

**Verification**

A **verifier** for a language $A$ is an algorithm $V$, where

$$A = \{w : V \text{ accepts } \langle w, c \rangle \text{ for some string } c\}.$$

We measure the time of a verifier only in terms of the length $w$, so a **polynomial time verifier** runs in polynomial time in the length of $w$. A language $A$ is **polynomial verifiable** if it has a polynomial time verifier.

# The Class NP

## Verification

A **verifier** for a language $A$ is an algorithm $V$, where

$$A = \{w : V \text{ accepts } \langle w, c \rangle \text{ for some string } c\}.$$

We measure the time of a verifier only in terms of the length $w$, so a **polynomial time verifier** runs in polynomial time in the length of $w$. A language $A$ is **polynomial verifiable** if it has a polynomial time verifier.

## The Class NP

**NP** is the class of language that have polynomial time verifiers.

# The Class NP

**Theorem 5**

*A language is in **NP** if and only if it is defined by some nondeterministic polynomial time Turing machine.*

# The Class NP

**Theorem 5**

*A language is in **NP** if and only if it is defined by some nondeterministic polynomial time Turing machine.*

**Nondeterministic Time**

$$\text{NTIME}(t(n)) = \{L : L \text{ is decided by a } O(t(n)) \text{ time NTM}\}$$

# The Class NP

**Theorem 5**

*A language is in **NP** if and only if it is defined by some nondeterministic polynomial time Turing machine.*

**Nondeterministic Time**

$$\text{NTIME}(t(n)) = \{L : L \text{ is decided by a } O(t(n)) \text{ time NTM}\}$$

**Corollary 6**

$$\boldsymbol{NP} = \bigcup_k \text{NTIME}(n^k).$$

# Some NP Problems

**Theorem 7**

CLIQUE *is in* ***NP****.*

# Some NP Problems

**Theorem 7**

CLIQUE *is in* ***NP****.*

- Given a collection of numbers $S = \{x_1, \ldots, x_k\}$ and a target number $t$
- Determine whether the collection contains a subcollection that adds up to $t$.

$$\text{SUBSET-SUM} = \{\langle S, t\rangle : \text{for some} \{y_1, \ldots, y_l\} \subseteq S, \sum y_i = t\}$$

**Theorem 8**

SUBSET-SUM *is in* ***NP****.*

# The P Versus NP Question

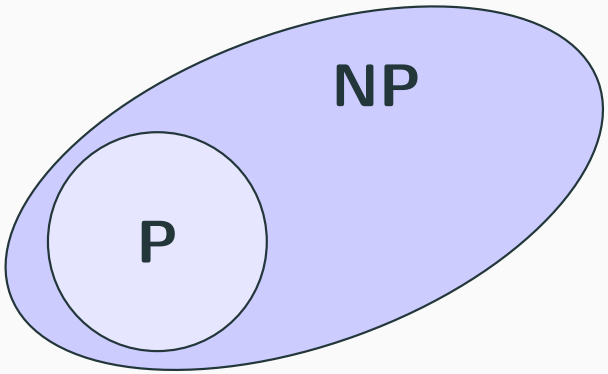
$$\mathbf{P} = \text{membership can be } \textit{decided} \text{ quickly}$$

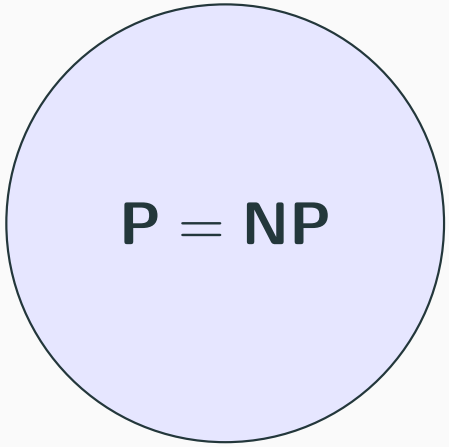
$$\mathbf{NP} = \text{membership can be } \textit{verified} \text{ quickly}$$

# The P Versus NP Question

**P** = membership can be *decided* quickly

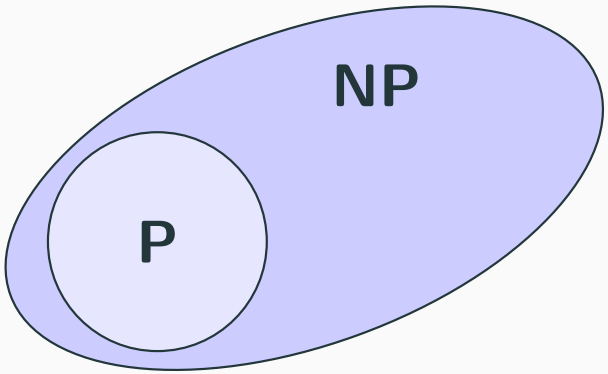
**NP** = membership can be *verified* quickly

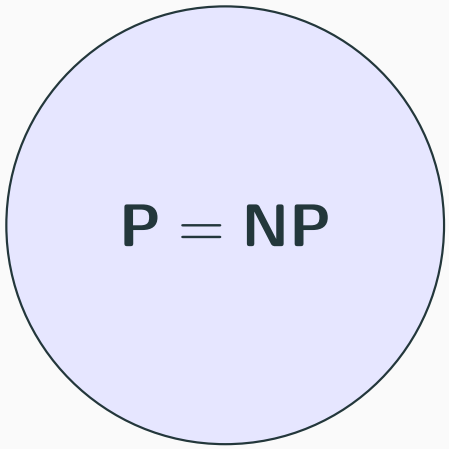


# The P Versus NP Question

$$\mathbf{P} = \text{membership can be } \textit{decided} \text{ quickly}$$
$$\mathbf{NP} = \text{membership can be } \textit{verified} \text{ quickly}$$




- We can prove that (best known results, so far)

$$\mathbf{NP} \subseteq \mathbf{EXPTIME} = \bigcup_k \text{TIME}\left(2^{n^k}\right)$$

# NP-Completeness

- There are certain problems in **NP** whose individual complexity is related to that of the entire class
- If a polynomial time algorithm exists for any of these problems, all problems in **NP** would be polynomial time solvable
- These problems are called **NP**-complete
- The phenomenon of **NP**-completeness is important for both theoretical and practical reasons
- The first **NP**-complete problem that we present is called the **satisfiability problem**

# NP-Completeness

- A Boolean formula is an expression involving Boolean variables and operations. For example,

$$\phi = (\overline{x} \wedge y) \vee (x \wedge \overline{z})$$

- A Boolean formula is *satisfiable* if some assignment of $0$s and $1$s to the variables makes the formula evaluate to $1$
- The preceding formula is satisfiable because the assignment $x = 0$, $y = 1$, and $z = 0$ makes $\phi$ evaluate to $1$
- We say the assignment satisfies $\phi$
- The *satisfiability problem* is to test whether a Boolean formula is satisfiable

# NP-Completeness

- Let $\text{SAT} = \{\langle \phi \rangle : \phi \text{ is a satisfiable Boolean formula}\}$

**Theorem 9**

$\text{SAT} \in \boldsymbol{P}$ *if and only if* $\boldsymbol{P} = \boldsymbol{NP}$.

# Polynomial Time Reducibility

- When a problem $A$ reduces to problem $B$
- A solution to $B$ can be used to solve $A$

# Polynomial Time Reducibility

- When a problem $A$ reduces to problem $B$
- A solution to $B$ can be used to solve $A$

**Poly-Time Computable Function**

A function $f : \Sigma^* \to \Sigma^*$ is a **polynomial time computable function** if some polynomial time Turing machine $M$ exists that halts with just $f(w)$ on its tape, when started on any input $w$.

# Polynomial Time Reducibility

## Poly-Time Redudicibility

Language $A$ is **polynomial time mapping reducible** or simply **polynomial time reducible** to language $B$, written $A \leq_{\mathrm{P}} B$, if a polynomial time computable function $f : \Sigma^* \to \Sigma^*$ exits, where for every $w$

$$w \in A \Leftrightarrow f(w) \in B.$$

The function $f$ is called the **polynomial time reduction** of $A$ to $B$.

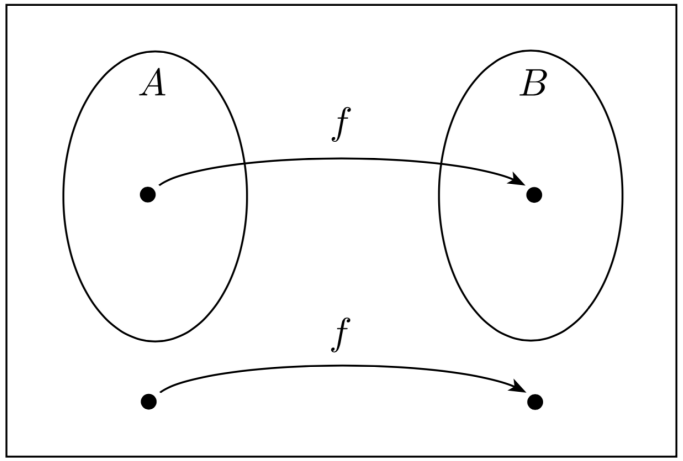
# Polynomial Time Reducibility

## Poly-Time Redudicibility

Language $A$ is **polynomial time mapping reducible** or simply **polynomial time reducible** to language $B$, written $A \leq_{\mathrm{P}} B$, if a polynomial time computable function $f : \Sigma^* \to \Sigma^*$ exits, where for every $w$

$$w \in A \Leftrightarrow f(w) \in B.$$

The function $f$ is called the **polynomial time reduction** of $A$ to $B$.



# Polynomial Time Reducibility

- As with an ordinary mapping reduction
- A polynomial time reduction of $A$ to $B$ provides a way to convert membership testing in $A$ to membership testing in $B$
- To test whether $w \in A$, we use the reduction $f$ to map $w$ to $f(w)$ and test whether $f(w) \in B$
- If one language is polynomial time reducible to a language already known to have a polynomial time solution, we obtain a polynomial time solution to the original language

# Polynomial Time Reducibility

- As with an ordinary mapping reduction
- A polynomial time reduction of $A$ to $B$ provides a way to convert membership testing in $A$ to membership testing in $B$
- To test whether $w \in A$, we use the reduction $f$ to map $w$ to $f(w)$ and test whether $f(w) \in B$
- If one language is polynomial time reducible to a language already known to have a polynomial time solution, we obtain a polynomial time solution to the original language

**Theorem 10**

*If $A \leq_{\mathrm{P}} B$ and $B \in \boldsymbol{P}$ then $A \in \boldsymbol{P}$.*

# Polynomial Time Reducibility

- A $3$-CNF formula has three literals in all clauses as in

$$(x_1 \vee \overline{x_2} \vee \overline{x_3}) \wedge (x_3 \vee \overline{x_5} \vee x_6) \wedge (x_4 \vee x_5 \vee x_6)$$

# Polynomial Time Reducibility

- A 3-CNF formula has three literals in all clauses as in

$$(x_1 \vee \overline{x_2} \vee \overline{x_3}) \wedge (x_3 \vee \overline{x_5} \vee x_6) \wedge (x_4 \vee x_5 \vee x_6)$$

**Theorem 11**

3-SAT *is polynomial time reducibile to* CLIQUE.

# Proof of Theorem 11

- Let $\phi$ be a formula with $k$ clauses

$$\phi = (a_1 \vee b_1 \vee c_1) \wedge (a_2 \vee b_2 \vee c_2) \wedge \cdots \wedge (a_k \vee b_k \vee c_k)$$

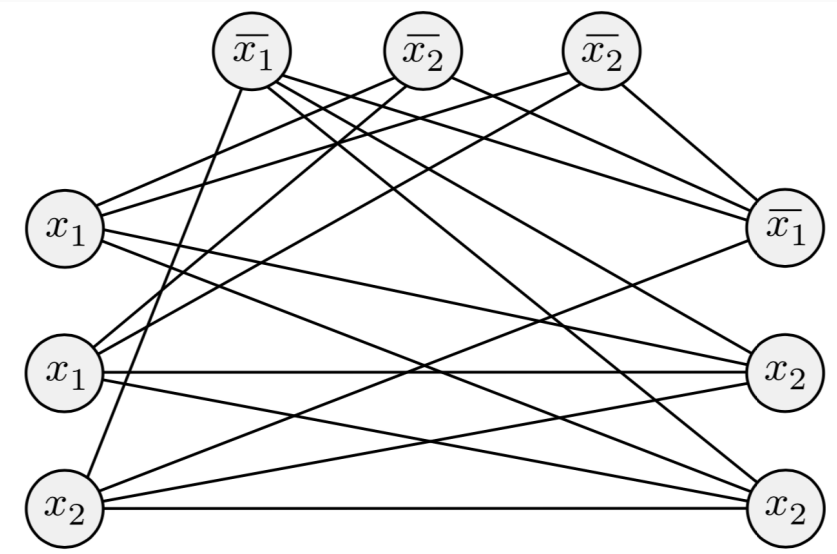
- The reduction $f$ generates the string $\langle G, k \rangle$ where $G$ is an undirected graph as following
- The nodes of $G$ are organized into $k$ groups of three nodes each called the **triples** $t_1, \ldots, t_k$ (each triple corresponds to one of the clauses in $\phi$ and each node in a triple corresponds to a literal in the associated clause)
- Each edge of $G$ connect all but two types of pairs of nodes in $G$
- No edge is present between nodes in the same triple, and no edge is present b/w two nodes with contradictory labels

# Example 1

- For example consider
  $\phi = (x_1 \vee x_1 \vee x_2) \wedge (\overline{x_1} \vee \overline{x_2} \vee \overline{x_2}) \wedge (\overline{x_1} \vee x_2 \vee x_2)$

# Example 1

- For example consider
  $\phi = (x_1 \vee x_1 \vee x_2) \wedge (\overline{x_1} \vee \overline{x_2} \vee \overline{x_2}) \wedge (\overline{x_1} \vee x_2 \vee x_2)$




- $\phi$ is satisfiable iff $G$ has a $k$-clique.

# Example 2

- Consider another example

$$\phi = (\overline{x_1} \vee x_2 \vee x_3) \wedge (x_1 \vee x_2 \vee \overline{x_3}) \wedge (x_1 \vee \overline{x_2} \vee x_3).$$

# Example 2

- Consider another example

$$\phi = (\overline{x_1} \vee x_2 \vee x_3) \wedge (x_1 \vee x_2 \vee \overline{x_3}) \wedge (x_1 \vee \overline{x_2} \vee x_3).$$

# Example 2

- Consider another example

$$\phi = (\overline{x_1} \vee x_2 \vee x_3) \wedge (x_1 \vee x_2 \vee \overline{x_3}) \wedge (x_1 \vee \overline{x_2} \vee x_3).$$


# Example 2

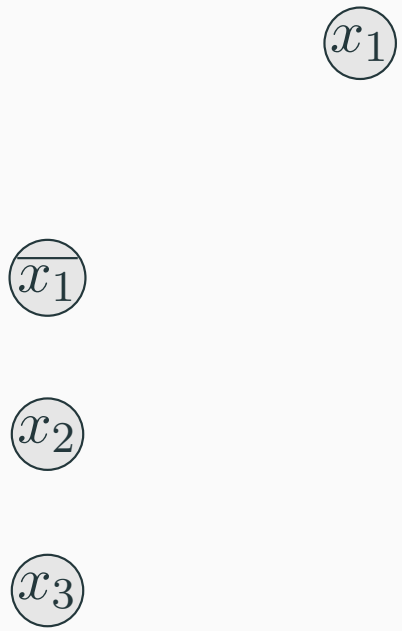
- Consider another example

$$\phi = (\overline{x_1} \vee x_2 \vee x_3) \wedge (x_1 \vee x_2 \vee \overline{x_3}) \wedge (x_1 \vee \overline{x_2} \vee x_3).$$

# Example 2

- Consider another example

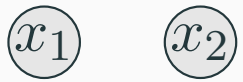
$$\phi = (\overline{x_1} \vee x_2 \vee x_3) \wedge (x_1 \vee x_2 \vee \overline{x_3}) \wedge (x_1 \vee \overline{x_2} \vee x_3).$$



# Example 2

- Consider another example

$$\phi = (\overline{x_1} \vee x_2 \vee x_3) \wedge (x_1 \vee x_2 \vee \overline{x_3}) \wedge (x_1 \vee \overline{x_2} \vee x_3).$$






# Example 2

- Consider another example

$$\phi = (\overline{x_1} \vee x_2 \vee x_3) \wedge (x_1 \vee x_2 \vee \overline{x_3}) \wedge (x_1 \vee \overline{x_2} \vee x_3).$$




# Example 2

- Consider another example

$$\phi = (\overline{x_1} \vee x_2 \vee x_3) \wedge (x_1 \vee x_2 \vee \overline{x_3}) \wedge (x_1 \vee \overline{x_2} \vee x_3).$$


# Example 2

- Consider another example

$$\phi = (\overline{x_1} \vee x_2 \vee x_3) \wedge (x_1 \vee x_2 \vee \overline{x_3}) \wedge (x_1 \vee \overline{x_2} \vee x_3).$$


# Example 2

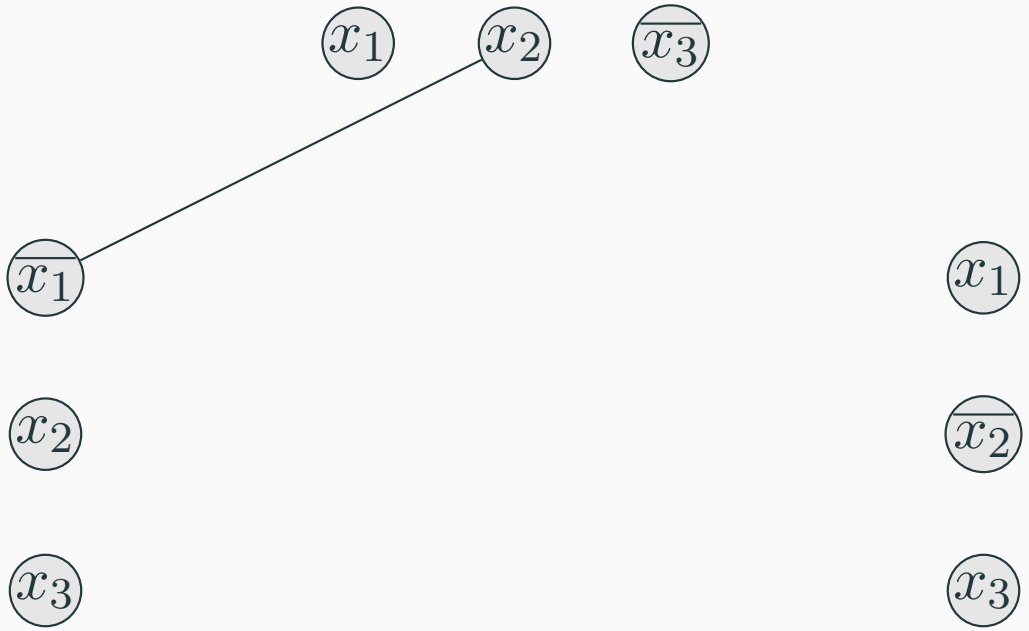
- Consider another example

$$\phi = (\overline{x_1} \vee x_2 \vee x_3) \wedge (x_1 \vee x_2 \vee \overline{x_3}) \wedge (x_1 \vee \overline{x_2} \vee x_3).$$


# Example 2

- Consider another example

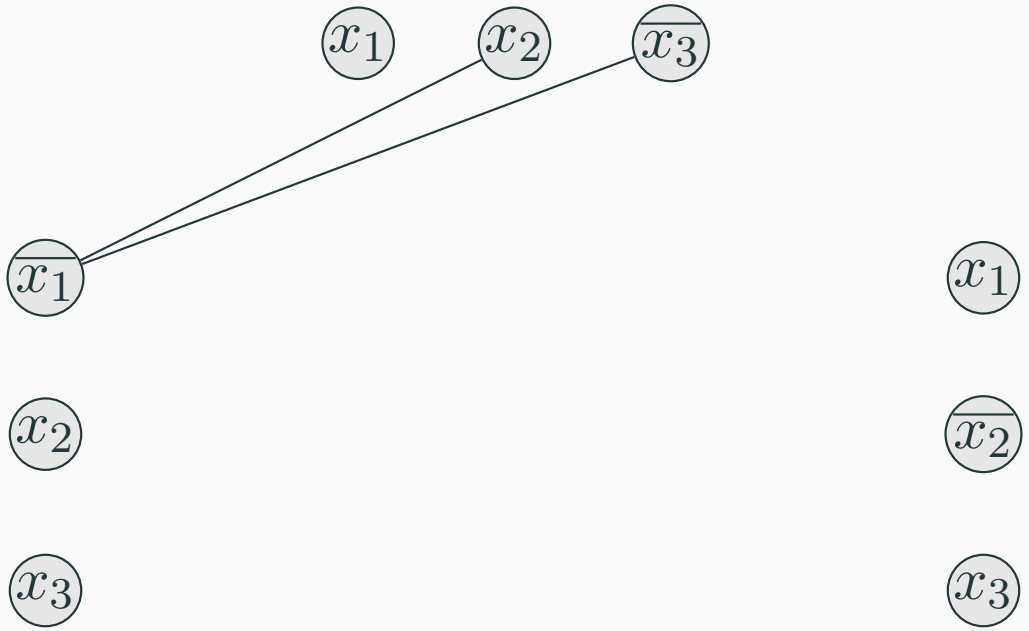
$$\phi = (\overline{x_1} \vee x_2 \vee x_3) \wedge (x_1 \vee x_2 \vee \overline{x_3}) \wedge (x_1 \vee \overline{x_2} \vee x_3).$$



# Example 2

- Consider another example

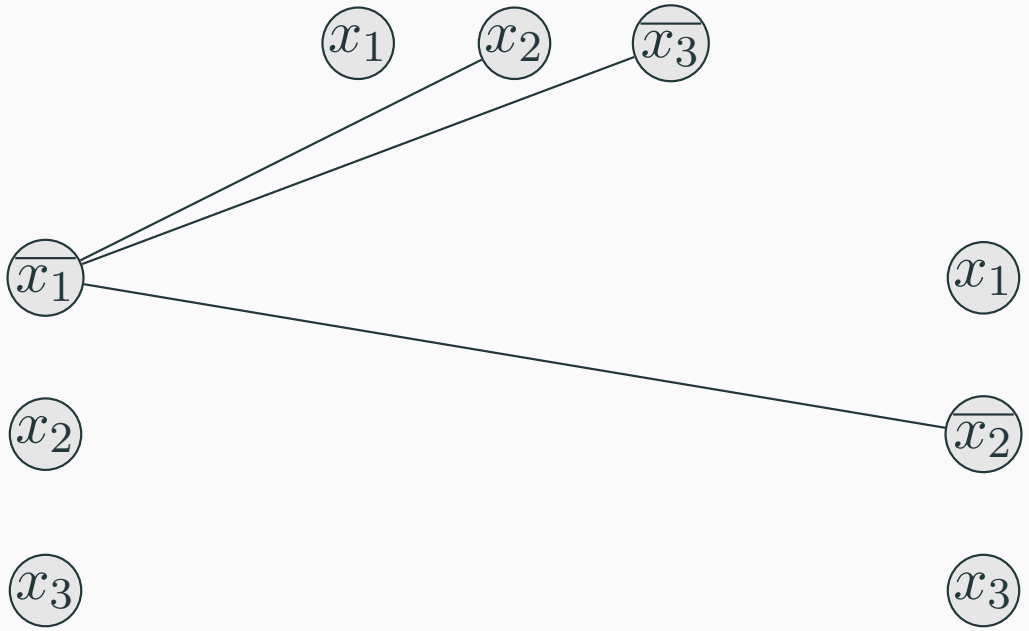
$$\phi = (\overline{x_1} \vee x_2 \vee x_3) \wedge (x_1 \vee x_2 \vee \overline{x_3}) \wedge (x_1 \vee \overline{x_2} \vee x_3).$$



# Example 2

- Consider another example

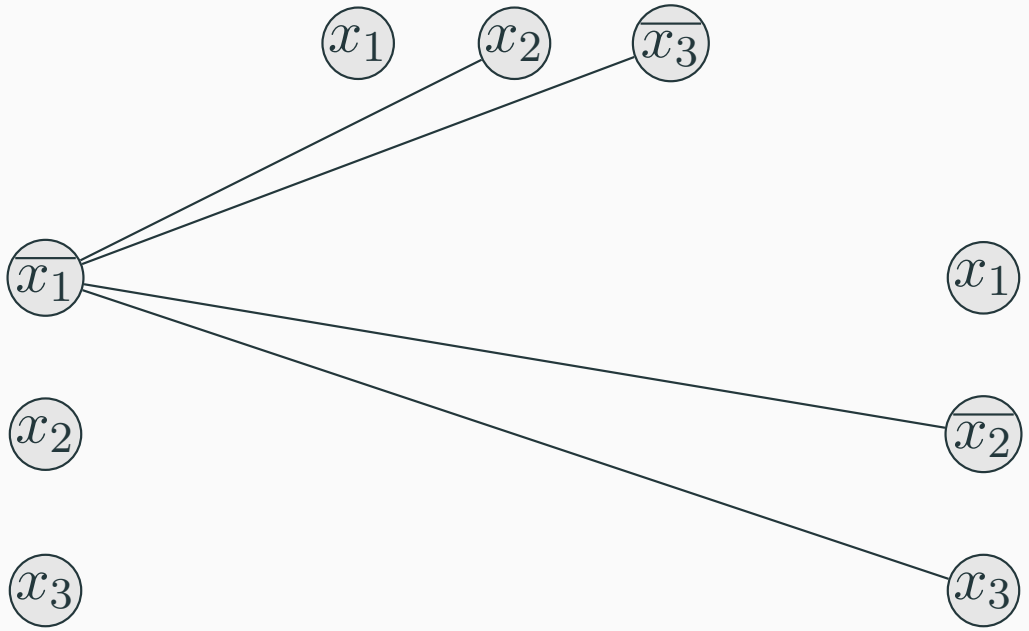
$$\phi = (\overline{x_1} \vee x_2 \vee x_3) \wedge (x_1 \vee x_2 \vee \overline{x_3}) \wedge (x_1 \vee \overline{x_2} \vee x_3).$$



# Example 2

- Consider another example

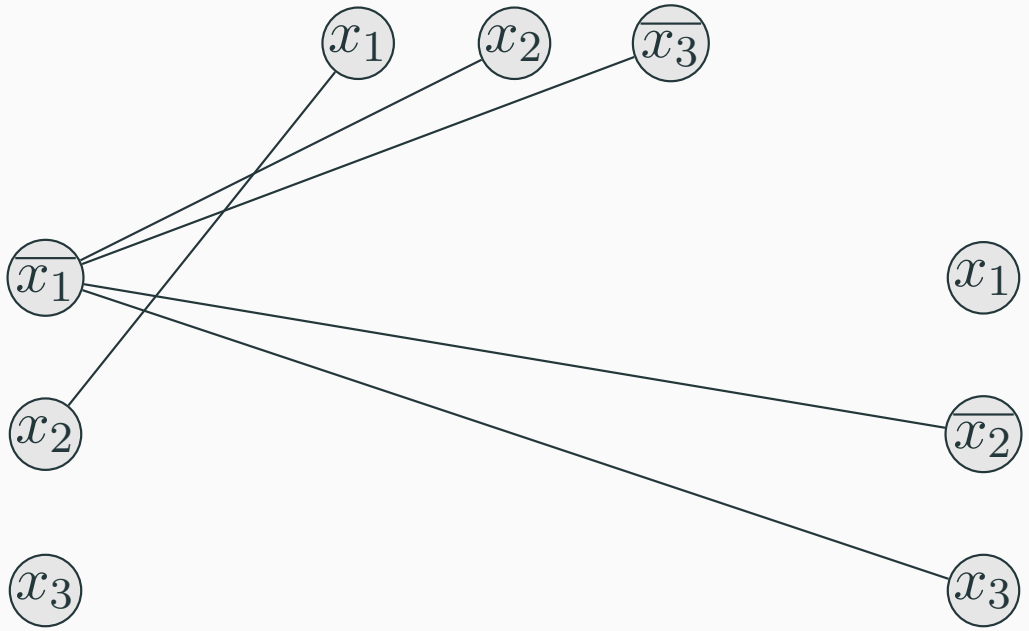
$$\phi = (\overline{x_1} \vee x_2 \vee x_3) \wedge (x_1 \vee x_2 \vee \overline{x_3}) \wedge (x_1 \vee \overline{x_2} \vee x_3).$$



# Example 2

- Consider another example

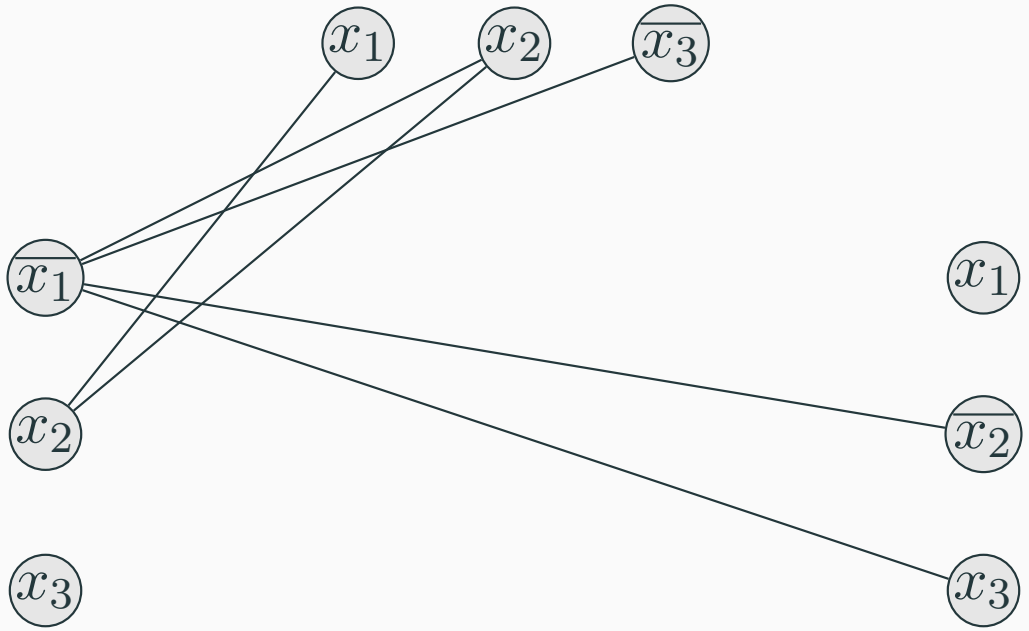
$$\phi = (\overline{x_1} \vee x_2 \vee x_3) \wedge (x_1 \vee x_2 \vee \overline{x_3}) \wedge (x_1 \vee \overline{x_2} \vee x_3).$$



# Example 2

- Consider another example

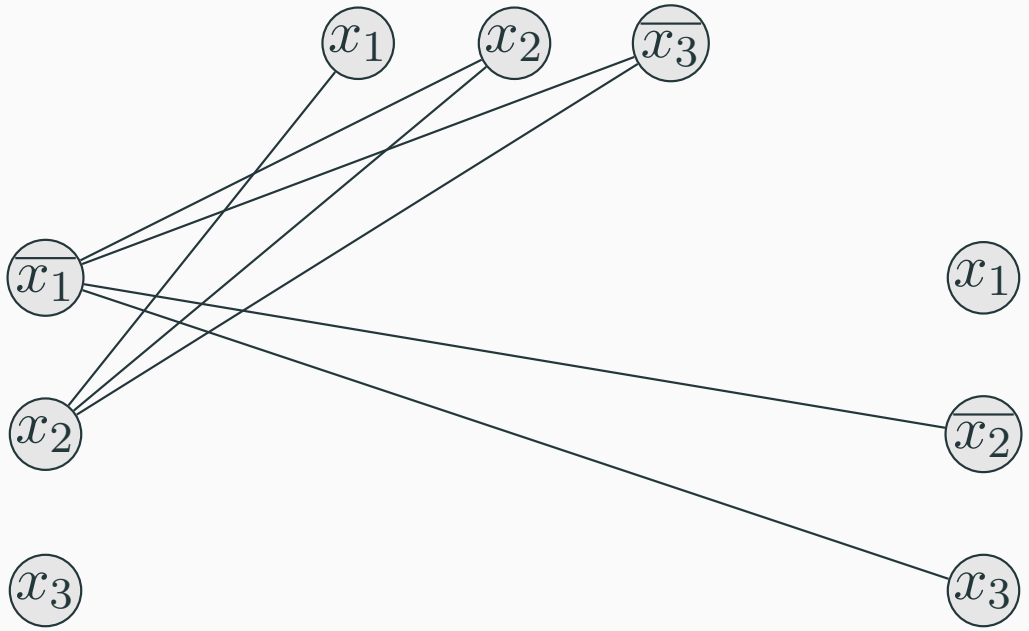
$$\phi = (\overline{x_1} \vee x_2 \vee x_3) \wedge (x_1 \vee x_2 \vee \overline{x_3}) \wedge (x_1 \vee \overline{x_2} \vee x_3).$$



# Example 2

- Consider another example

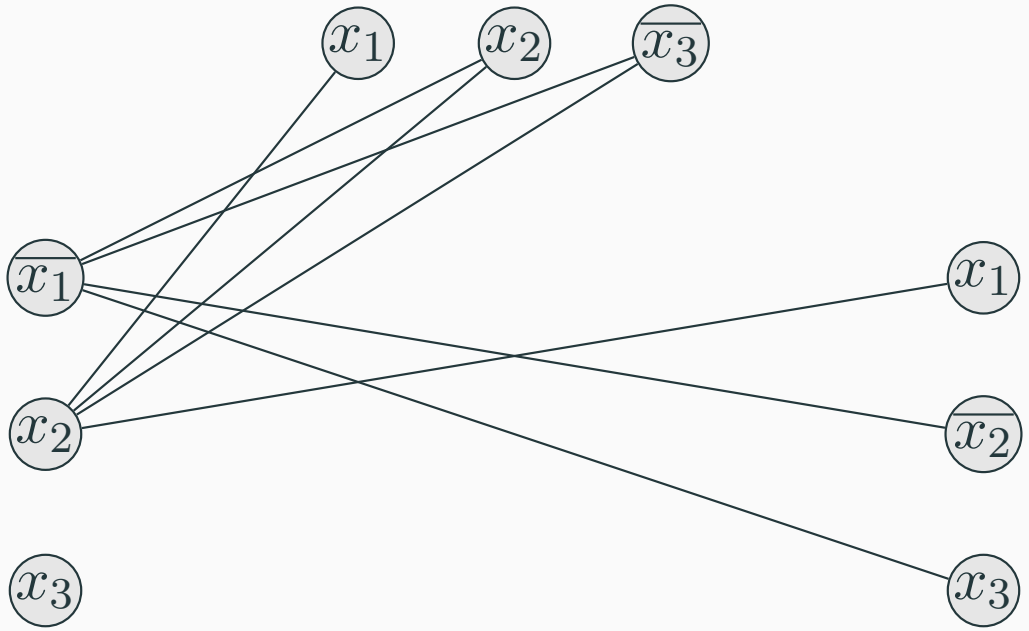
$$\phi = (\overline{x_1} \vee x_2 \vee x_3) \wedge (x_1 \vee x_2 \vee \overline{x_3}) \wedge (x_1 \vee \overline{x_2} \vee x_3).$$



# Example 2

- Consider another example

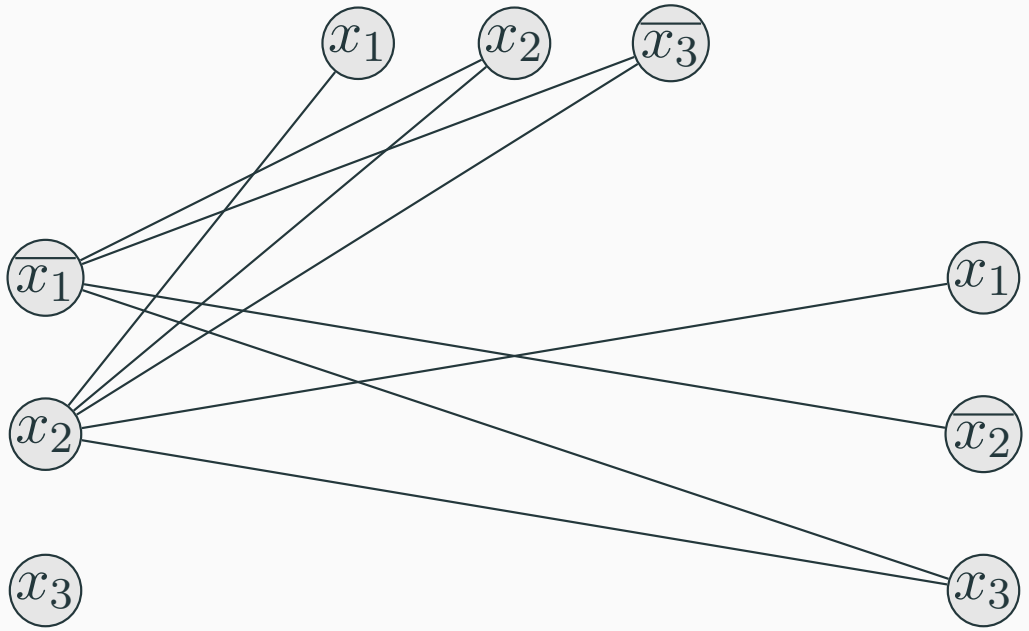
$$\phi = (\overline{x_1} \vee x_2 \vee x_3) \wedge (x_1 \vee x_2 \vee \overline{x_3}) \wedge (x_1 \vee \overline{x_2} \vee x_3).$$



# Example 2

- Consider another example

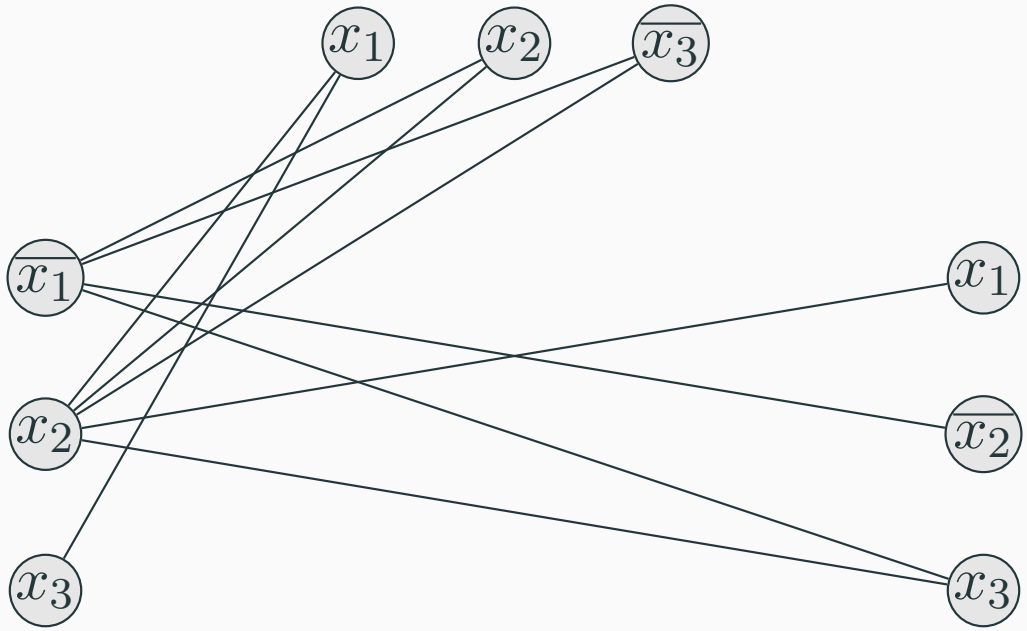
$$\phi = (\overline{x_1} \vee x_2 \vee x_3) \wedge (x_1 \vee x_2 \vee \overline{x_3}) \wedge (x_1 \vee \overline{x_2} \vee x_3).$$



# Example 2

- Consider another example

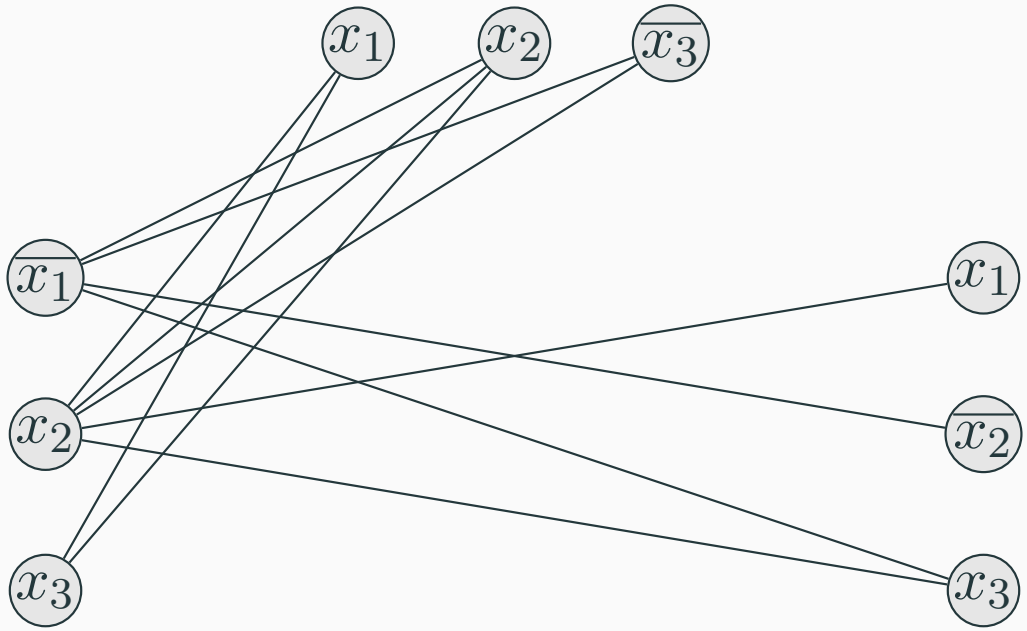
$$\phi = (\overline{x_1} \vee x_2 \vee x_3) \wedge (x_1 \vee x_2 \vee \overline{x_3}) \wedge (x_1 \vee \overline{x_2} \vee x_3).$$



# Example 2

- Consider another example

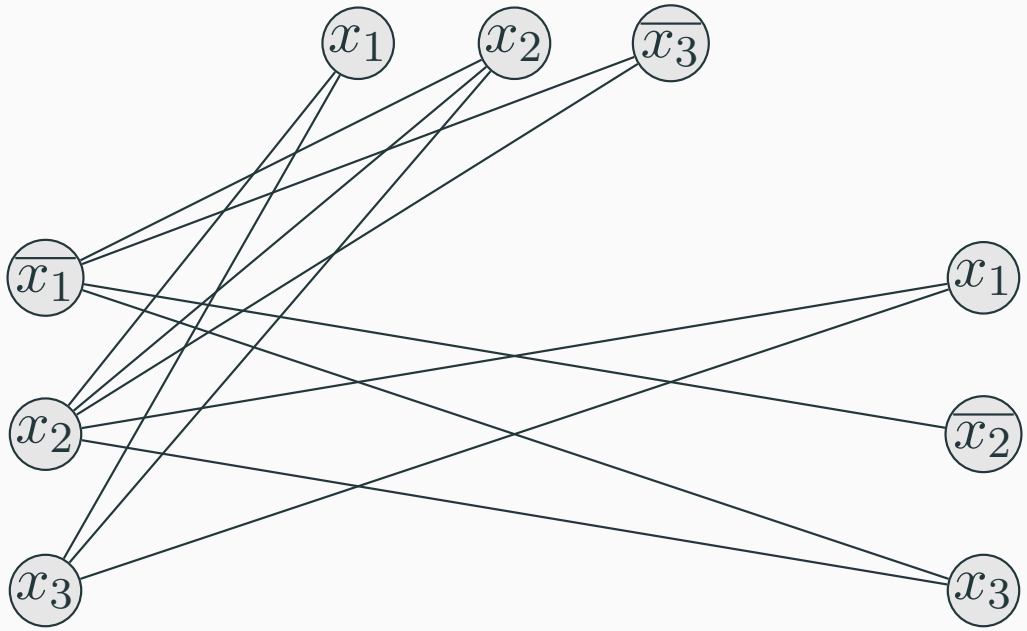
$$\phi = (\overline{x_1} \vee x_2 \vee x_3) \wedge (x_1 \vee x_2 \vee \overline{x_3}) \wedge (x_1 \vee \overline{x_2} \vee x_3).$$



# Example 2

- Consider another example

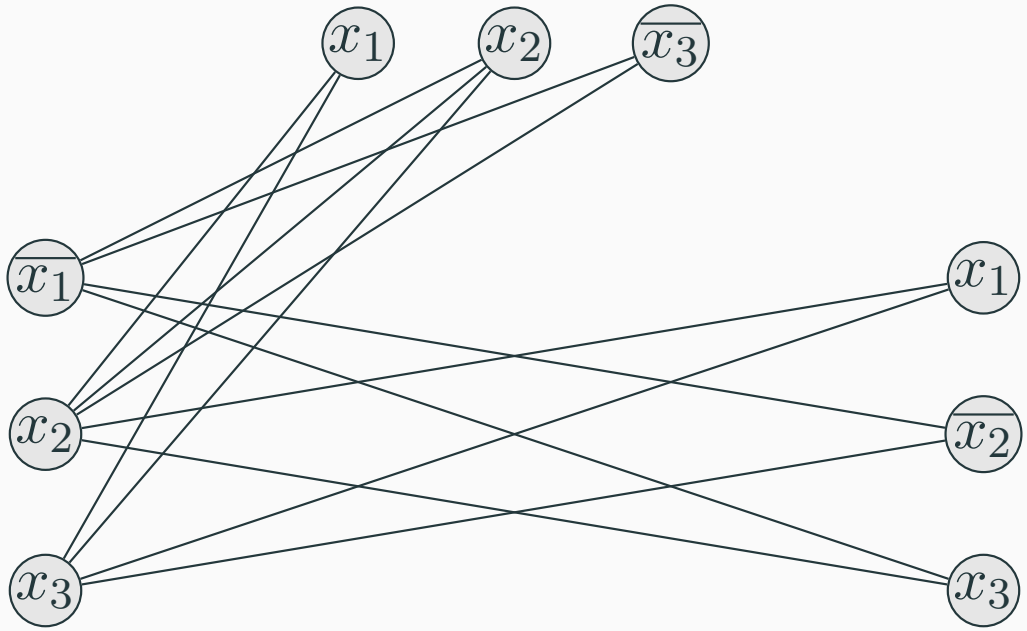
$$\phi = (\overline{x_1} \vee x_2 \vee x_3) \wedge (x_1 \vee x_2 \vee \overline{x_3}) \wedge (x_1 \vee \overline{x_2} \vee x_3).$$



# Example 2

- Consider another example

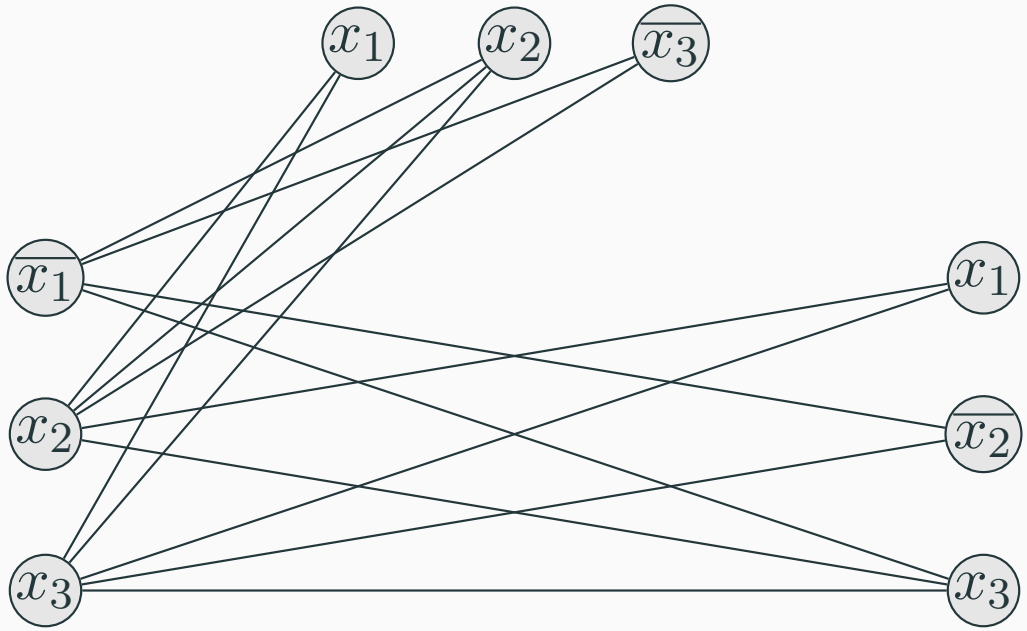
$$\phi = (\overline{x_1} \vee x_2 \vee x_3) \wedge (x_1 \vee x_2 \vee \overline{x_3}) \wedge (x_1 \vee \overline{x_2} \vee x_3).$$



# Example 2

- Consider another example

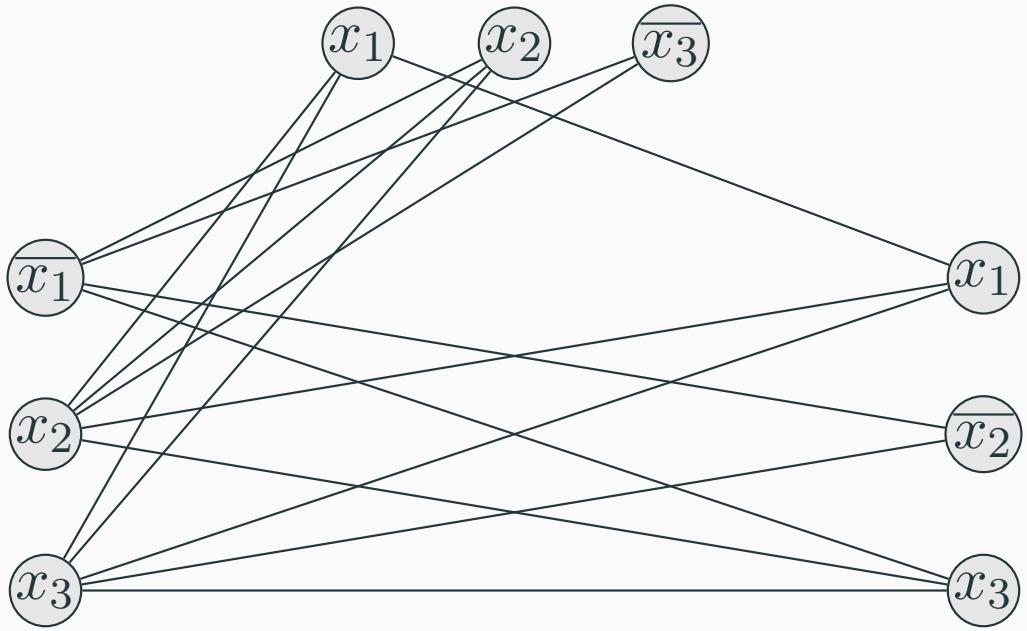
$$\phi = (\overline{x_1} \vee x_2 \vee x_3) \wedge (x_1 \vee x_2 \vee \overline{x_3}) \wedge (x_1 \vee \overline{x_2} \vee x_3).$$



# Example 2

- Consider another example

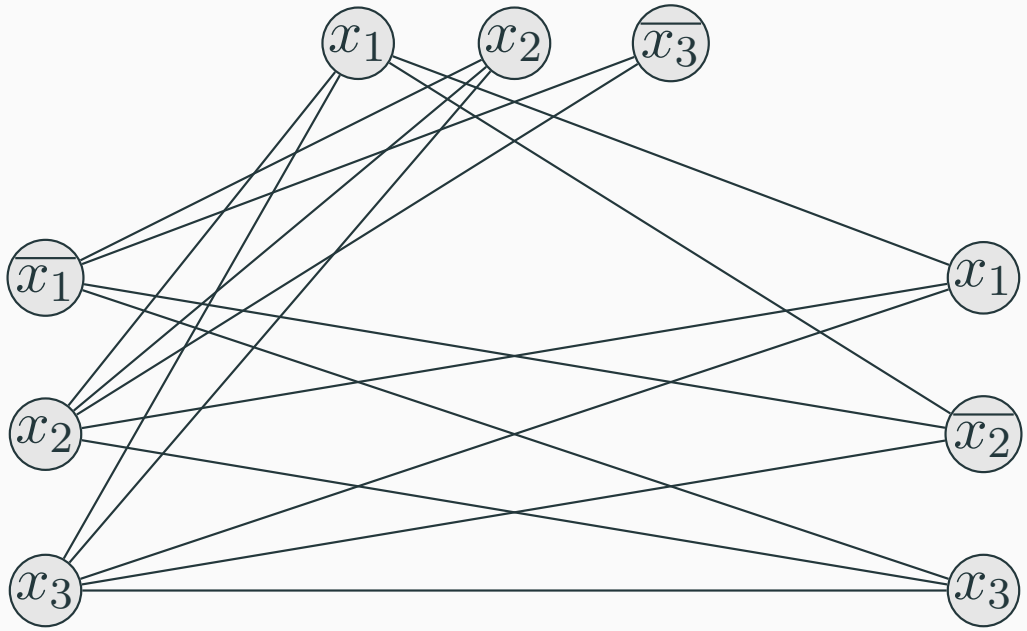
$$\phi = (\overline{x_1} \vee x_2 \vee x_3) \wedge (x_1 \vee x_2 \vee \overline{x_3}) \wedge (x_1 \vee \overline{x_2} \vee x_3).$$



# Example 2

- Consider another example

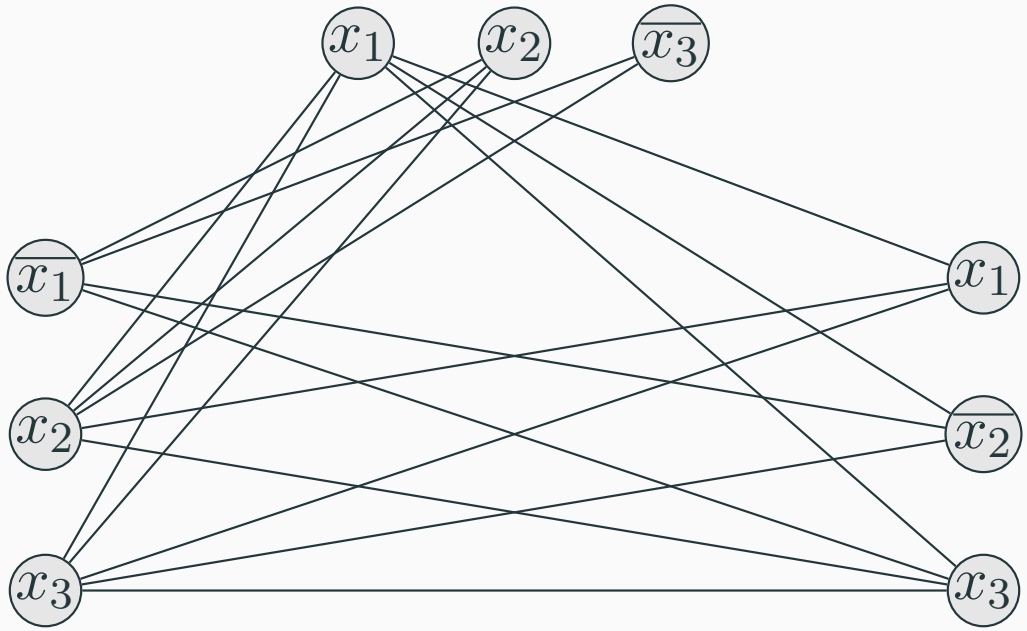
$$\phi = (\overline{x_1} \vee x_2 \vee x_3) \wedge (x_1 \vee x_2 \vee \overline{x_3}) \wedge (x_1 \vee \overline{x_2} \vee x_3).$$



# Example 2

- Consider another example

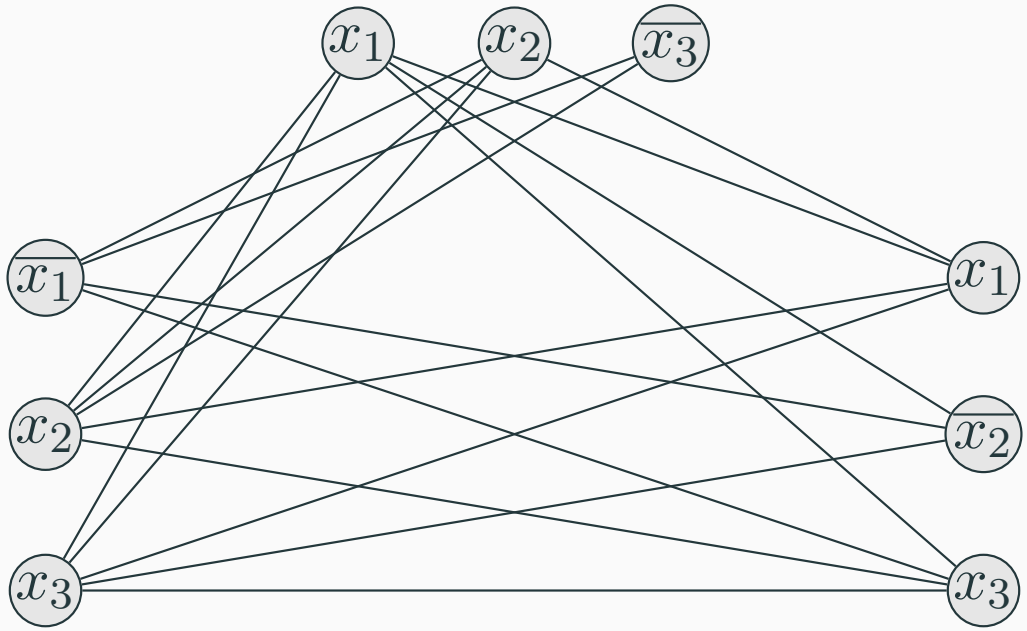
$$\phi = (\overline{x_1} \vee x_2 \vee x_3) \wedge (x_1 \vee x_2 \vee \overline{x_3}) \wedge (x_1 \vee \overline{x_2} \vee x_3).$$



# Example 2

- Consider another example

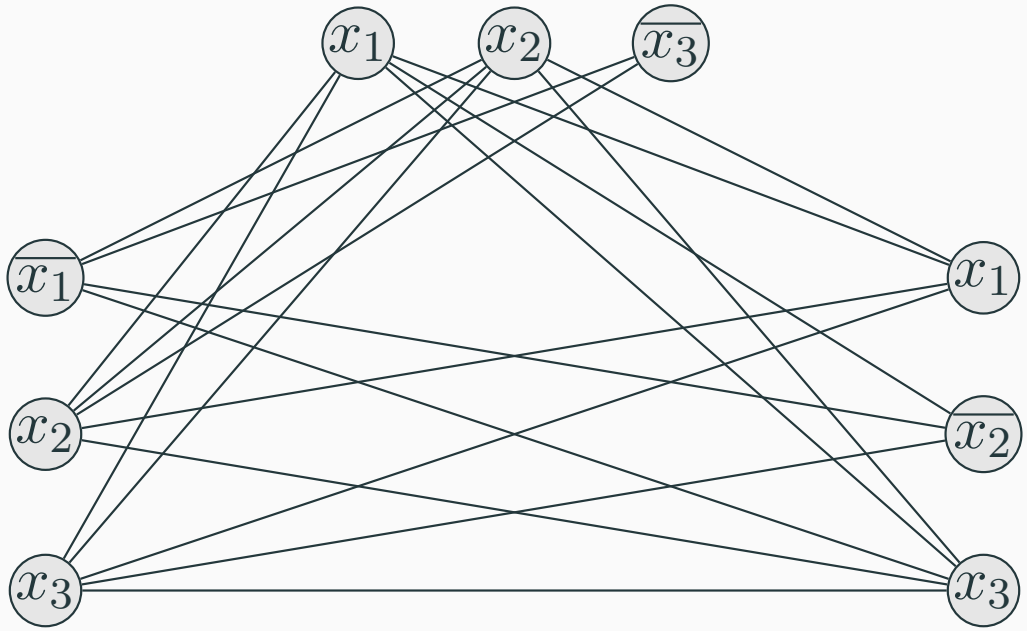
$$\phi = (\overline{x_1} \vee x_2 \vee x_3) \wedge (x_1 \vee x_2 \vee \overline{x_3}) \wedge (x_1 \vee \overline{x_2} \vee x_3).$$



# Example 2

- Consider another example

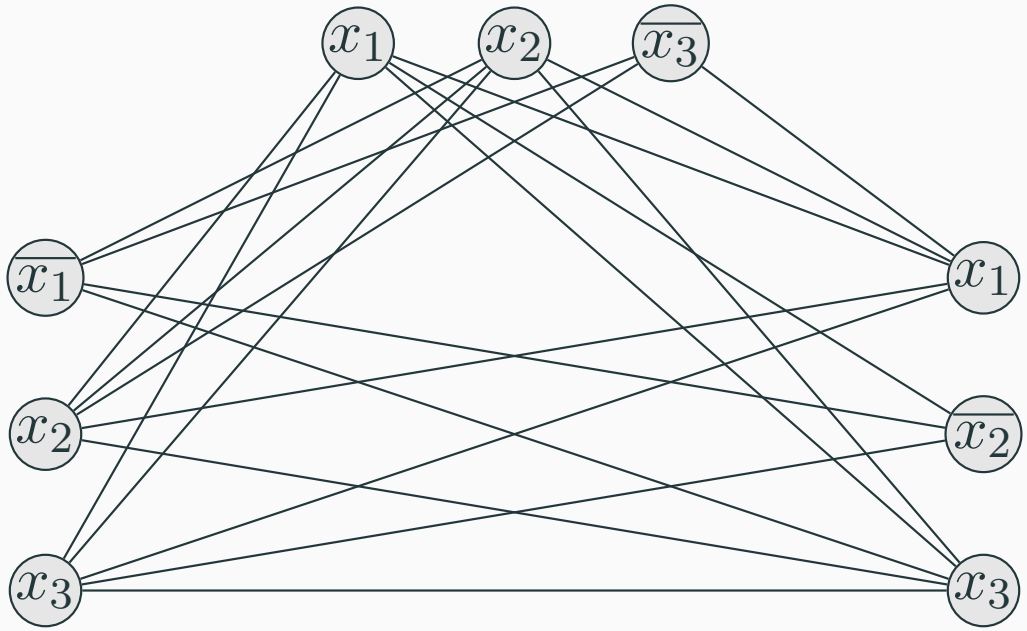
$$\phi = (\overline{x_1} \vee x_2 \vee x_3) \wedge (x_1 \vee x_2 \vee \overline{x_3}) \wedge (x_1 \vee \overline{x_2} \vee x_3).$$



# Example 2

- Consider another example

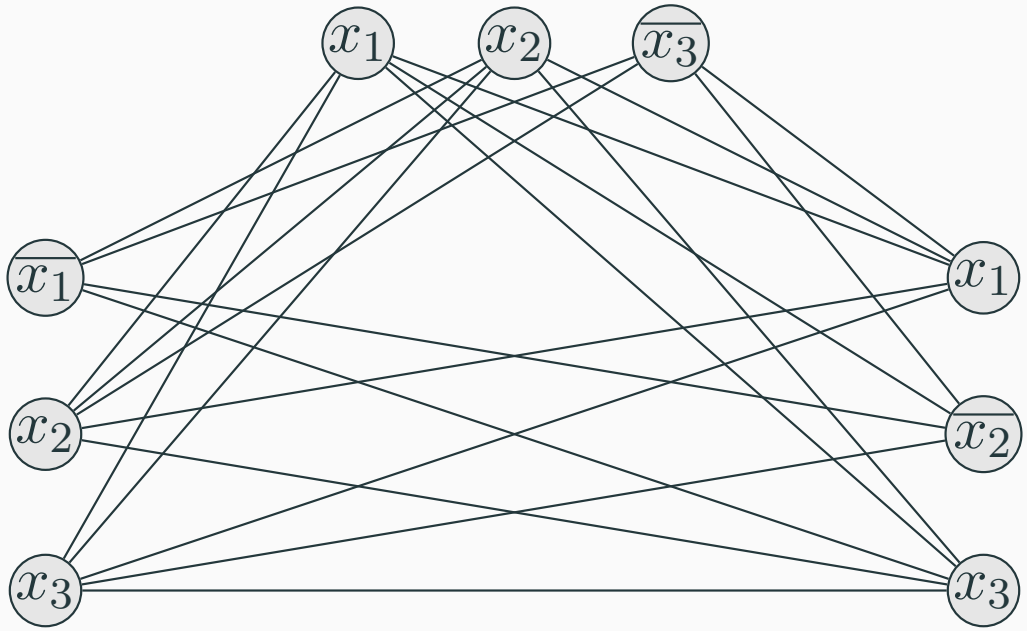
$$\phi = (\overline{x_1} \vee x_2 \vee x_3) \wedge (x_1 \vee x_2 \vee \overline{x_3}) \wedge (x_1 \vee \overline{x_2} \vee x_3).$$



# Example 2

- Consider another example

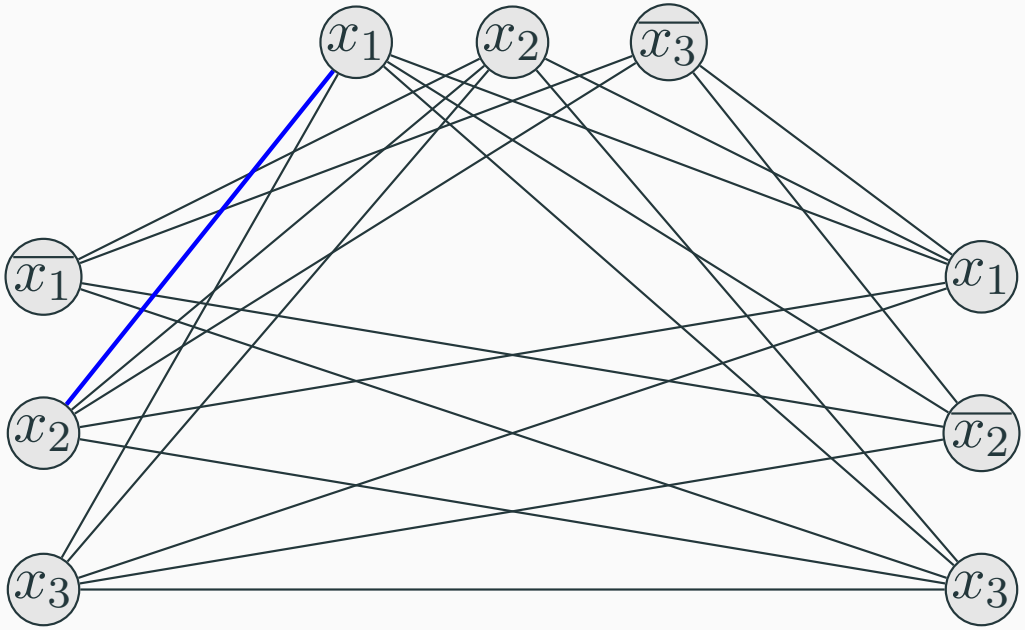
$$\phi = (\overline{x_1} \vee x_2 \vee x_3) \wedge (x_1 \vee x_2 \vee \overline{x_3}) \wedge (x_1 \vee \overline{x_2} \vee x_3).$$



# Example 2

- Consider another example

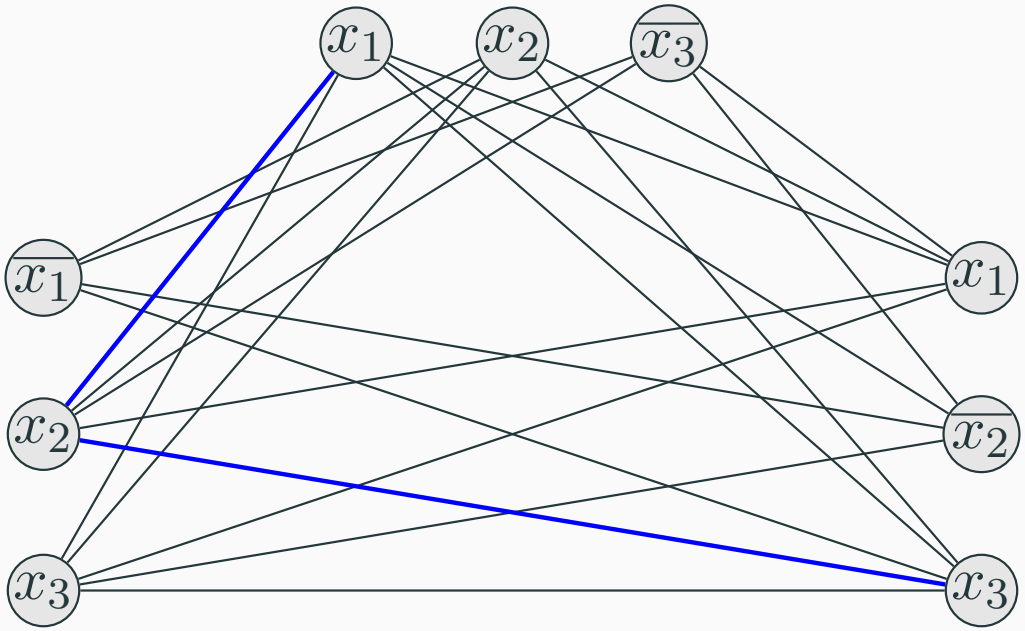
$$\phi = (\overline{x_1} \vee x_2 \vee x_3) \wedge (x_1 \vee x_2 \vee \overline{x_3}) \wedge (x_1 \vee \overline{x_2} \vee x_3).$$



# Example 2

- Consider another example

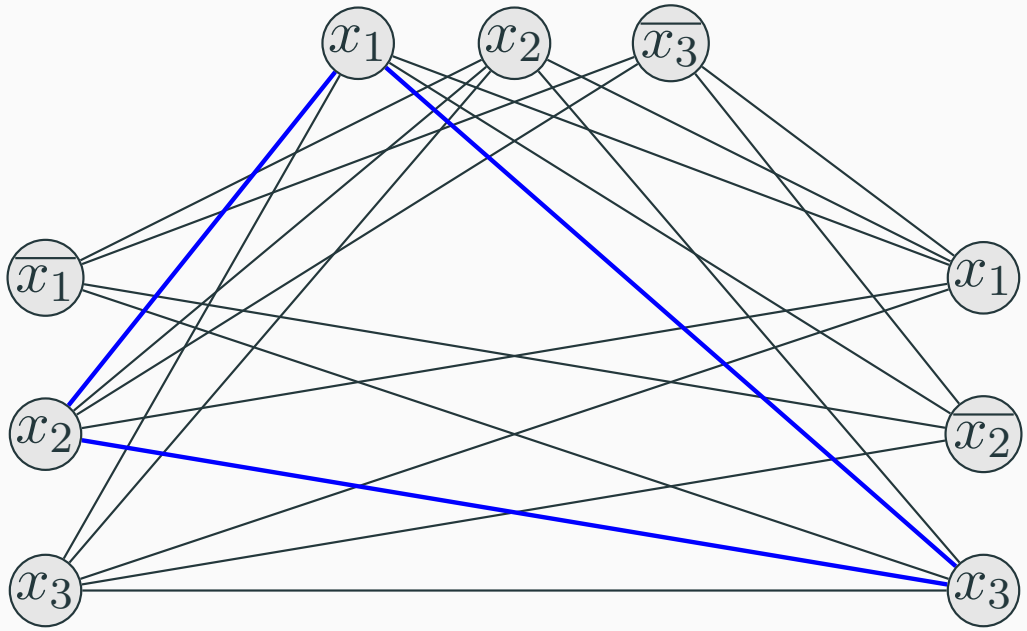
$$\phi = (\overline{x_1} \vee x_2 \vee x_3) \wedge (x_1 \vee x_2 \vee \overline{x_3}) \wedge (x_1 \vee \overline{x_2} \vee x_3).$$



# Example 2

- Consider another example

$$\phi = (\overline{x_1} \vee x_2 \vee x_3) \wedge (x_1 \vee x_2 \vee \overline{x_3}) \wedge (x_1 \vee \overline{x_2} \vee x_3).$$



# NP-Completeness

**NP-Completeness**

A language $B$ is **NP**-complete if it satisfies two conditions:

1. $B$ is in **NP**, and
2. every $A$ in **NP** is polynomial time reducible to $B$.

# NP-Completeness

## NP-Completeness

A language $B$ is **NP**-complete if it satisfies two conditions:

1. $B$ is in **NP**, and
2. every $A$ in **NP** is polynomial time reducible to $B$.

## Theorem 12

*If $B$ is **NP**-complete and $B \in$ **P**, then **P** = **NP**.*

# NP-Completeness

**Theorem 13**

*If $B$ is **NP**-complete and $B \leq_P C$ for $C$ in **NP**, then $C$ is **NP**-complete.*

# NP-Completeness

**Theorem 13**

*If $B$ is **NP**-complete and $B \leq_P C$ for $C$ in **NP**, then $C$ is **NP**-complete.*

**Proof.**

- $C$ is in **NP**
- We need to show that every $A$ in **NP** is poly. time reducible to $C$
- Since $B$ is **NP**-complete, every language in **NP** is poly. time reducible to $B$, and $B$ is poly. time reducible to $C$
- $A \leq_P B$ and $B \leq_P C$ then $A \leq_P C$
- Hence every language in **NP** is poly. tiime reducible to $C$.

□

# The Cook-Levin Theorem

**Theorem 14**

SAT *is* ***NP****-complete.*

# The Cook-Levin Theorem

**Theorem 14**

SAT *is* ***NP****-complete.*

**Corollary 15**

*3-*SAT *is* ***NP****-complete.*

# The Cook-Levin Theorem

**Theorem 14**

SAT *is* ***NP****-complete.*

**Corollary 15**

*3-*SAT *is* ***NP****-complete.*

# More NP-complete Problems

**Corollary 16**

CLIQUE *is* ***NP****-complete.*

# More NP-complete Problems

**Corollary 16**

CLIQUE *is* ***NP****-complete.*

**Vertex-Cover**

If $G$ is an undirected graph, **vertex cover** of $G$ is a subset of the nodes where every edge of $G$ touches one of those nodes.

$$\text{VERTEX-COVER} = \{\langle G, k\rangle : G \text{ has a } k\text{-node vertex cover}\}.$$

# More NP-complete Problems

**Corollary 16**

CLIQUE *is* ***NP****-complete.*

**Vertex-Cover**

If $G$ is an undirected graph, **vertex cover** of $G$ is a subset of the nodes where every edge of $G$ touches one of those nodes.

$$\text{VERTEX-COVER} = \{\langle G, k\rangle : G \text{ has a } k\text{-node vertex cover}\}.$$

**Theorem 17**

VERTEX-COVER *is* ***NP****-complete.*

# Proof of vertex-cover is NP-complete

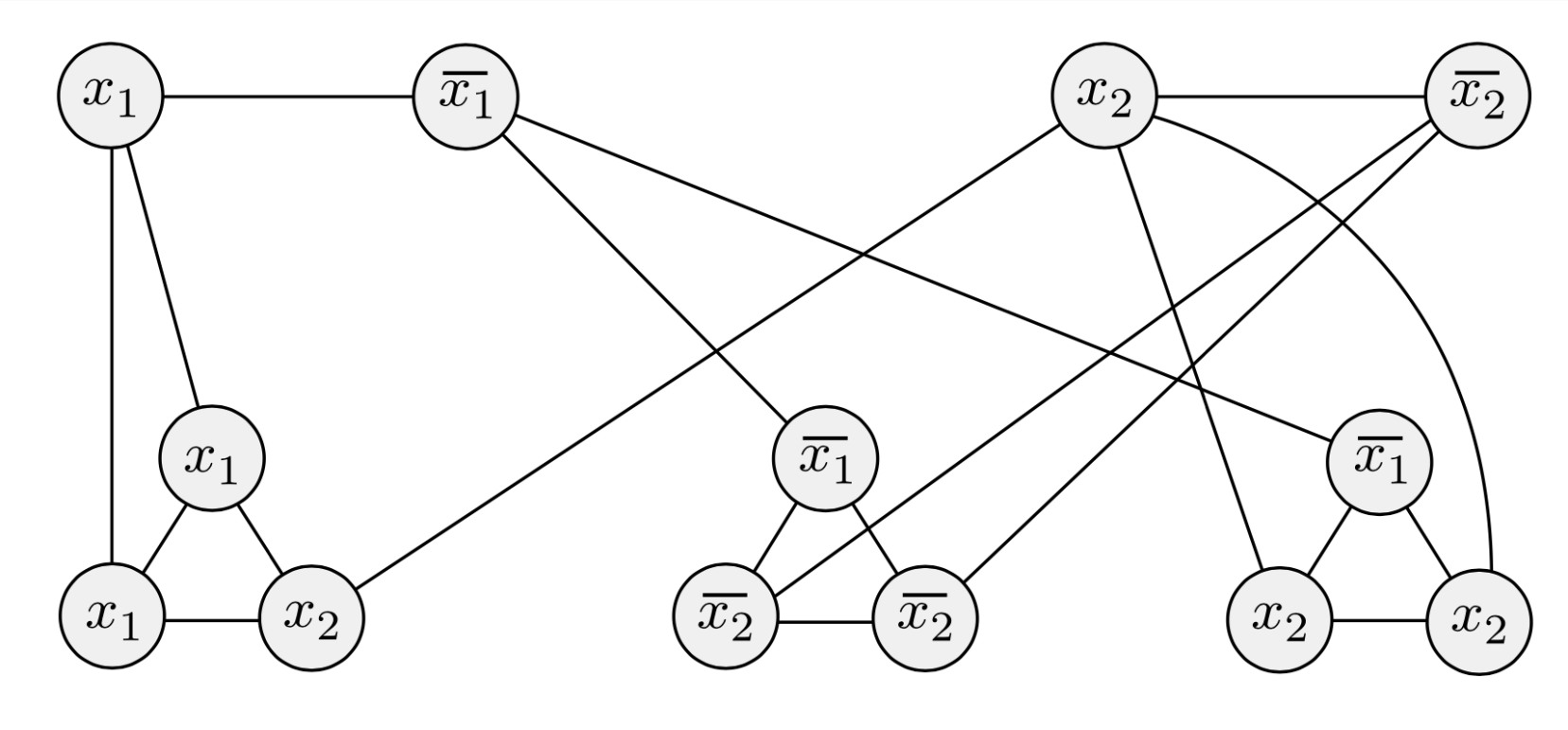
- VERTEX-COVER is in **NP**, 3-SAT $\leq_P$ VERTEX-COVER
- Let $\phi$ be a Boolean formula with $l$ clauses
- We construct a graph $G$ for $\phi$
- For each variable $x$ in each clause in $\phi$ we create a vertex in $G$ (triple)
- Each vertex (3) in a clause is connected to each other vertex in the clause
- We create additional vertices for each variable and its negation from $\phi$
- These additional vertices are connected to their copies in each triple
- These additional vertices are connected to their complement from the set of additional vertices
- $\phi$ is satisfiable iff $G$ has a vertex cover of $k = m + 2l$, where $l$ is no. of clauses and $m$ is the no. of variables.

# Example

- $\phi = (x_1 \vee x_1 \vee x_2) \wedge (\overline{x_1} \vee \overline{x_2} \vee \overline{x_2}) \wedge (\overline{x_1} \vee x_2 \vee x_2)$ and $k = m + 2l = 2 + 3(2) = 8$

# Example

- $\phi = (x_1 \vee x_1 \vee x_2) \wedge (\overline{x_1} \vee \overline{x_2} \vee \overline{x_2}) \wedge (\overline{x_1} \vee x_2 \vee x_2)$ and $k = m + 2l = 2 + 3(2) = 8$



## NP-Hardness

A language $B$ is **NP**-hard if:

- every $A$ in **NP** is polynomial time reducible to $B$.

## NP-Hardness

A language $B$ is **NP**-hard if:

- every $A$ in **NP** is polynomial time reducible to $B$.

We can rewrite above definition as following as well:

A language $B$ is **NP**-hard if some **NP**-complete language $A$ reduces to $B$ in polynomial time.

# NP-Hardness

## NP-Hardness

A language $B$ is **NP**-hard if:

- every $A$ in **NP** is polynomial time reducible to $B$.

We can rewrite above definition as following as well:

A language $B$ is **NP**-hard if some **NP**-complete language $A$ reduces to $B$ in polynomial time.

This means we can redefine **NP**-complete as following:

## NP-completeness

A language $B$ is **NP**-complete if:

1. $B$ is in **NP**, and
2. $B$ is **NP**-hard.